LOVELY PROFESSIONAL UNIVERSITY

Department of Computer Science & Engineering

PROJECT REPORT

Disk Scheduling Algorithm Visualizer

An Interactive Web-Based Simulation Tool

A comprehensive implementation and analysis of FCFS, SSTF, SCAN, and C-SCAN
disk scheduling algorithms with real-time visualization

PROJECT DETAILS

Student Name: Kumkum Kumari

Roll Number: 57

Course: B.Tech Computer Science and Engineering

Semester: 4th Semester

Subject: Operating Systems

Guide: Rizwan Sir

Submitted On: 22nd April

TABLE OF CONTENTS

1. Abstract 3
2. Introduction 3
   2.1 Background 3
   2.2 Motivation 3
3. Objectives 4
4. Literature Review 4
5. Methodology 5
6. Algorithm Explanations 5
   6.1 FCFS – First Come First Serve 5
   6.2 SSTF – Shortest Seek Time First 6
   6.3 SCAN – Elevator Algorithm 6
   6.4 C-SCAN – Circular SCAN 7
7. System Design 7
8. Implementation Details 8
9. Screenshots 9
10. Results & Analysis 10
11. Advantages & Limitations 11
12. Future Scope 12
13. Conclusion 12
14. References 13

1. ABSTRACT

*This project presents the design and implementation of a **Disk Scheduling Algorithm Visualizer** — an interactive, web-based educational tool that demonstrates the working principles of four fundamental disk scheduling algorithms: First Come First Serve (FCFS), Shortest Seek Time First (SSTF), SCAN (Elevator), and Circular SCAN (C-SCAN). The tool provides animated step-by-step visualization of disk head movement, real-time performance metrics (total seek time, throughput, number of movements), and a side-by-side comparison mode with bar charts and data tables. Implemented entirely using HTML, CSS, and vanilla JavaScript with Chart.js for data visualization, the project serves as a practical demonstration of operating system concepts related to disk I/O management, making abstract algorithms intuitive and accessible for computer science students.*

Keywords: Disk Scheduling, FCFS, SSTF, SCAN, C-SCAN, Operating Systems, Visualization, JavaScript, Algorithm Simulation

2. INTRODUCTION

2.1 Background

Hard disk drives (HDDs) are among the most widely used secondary storage devices in computer systems. The physical movement of the read/write head across the disk surface to access requested tracks introduces latency known as *seek time*. In a modern operating system, multiple processes simultaneously compete for disk access, resulting in a queue of pending I/O requests. The order in which these requests are processed directly determines the total seek time and, consequently, the overall system throughput.

Disk scheduling algorithms govern the order in which queued requests are serviced. A well-chosen algorithm can dramatically reduce the total distance traveled by the disk head, improving both performance and the lifespan of mechanical components. The study of these algorithms is a cornerstone topic in any Operating Systems course.

2.2 Motivation

While theoretical descriptions of disk scheduling algorithms are well-documented in textbooks, students often find it difficult to intuitively understand how the head moves and why one

algorithm outperforms another in specific scenarios. Existing tools tend to be either too simplistic (only showing final seek times) or too complex for beginners. This project was motivated by the need for a visually rich, interactive, and beginner-friendly simulation environment that bridges the gap between theory and practical understanding.

3. OBJECTIVES

- Implement all four major disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) with correct logic.
- Build an animated visualization showing step-by-step head movement across disk tracks.
- Calculate and display performance metrics: total seek time, number of movements, and throughput.
- Provide a comparison mode with bar charts and tabular data to allow easy evaluation of algorithms.
- Implement a "Best Algorithm Suggestion" feature that identifies the optimal algorithm for a given input.
- Design a professional, modern UI with dark theme suitable for educational and demonstration purposes.
- Allow user-configurable inputs: disk size, head position, request queue, and direction.
- Include random input generation for quick exploration of algorithm behavior.

4. LITERATURE REVIEW

Disk scheduling has been studied extensively since the early days of computing. Denning (1967) first formalized the concept of disk scheduling and identified seek time as a primary bottleneck in I/O performance. The classical algorithms — FCFS, SSTF, SCAN, and their variants — were developed and analyzed over the following decades.

Silberschatz, Galvin, and Gagne (2018) in their landmark textbook *Operating System Concepts* provide a comprehensive mathematical treatment of these algorithms and demonstrate through worked examples that SSTF reduces average seek time compared to FCFS, though it can lead to starvation of distant requests. They further show that SCAN and C-SCAN offer better fairness while achieving seek times approaching the optimal.

Tanenbaum and Bos (2014) in *Modern Operating Systems* extend this analysis to include the impact of rotational latency and disk geometry on algorithm performance in real hardware, noting that purely seek-time-based algorithms like SSTF and SCAN may not always be optimal when rotational latency is considered.

More recent work by Chen et al. (2019) investigates hybrid scheduling approaches combining SSTF's proximity optimization with SCAN's fairness guarantees, proposing algorithms for modern SSDs where seek time differences are negligible compared to HDDs. However, for educational purposes, the classical algorithms remain the most instructive models.

In the domain of educational visualization, several tools have been proposed. Scheffer et al. (2020) demonstrated that interactive simulation tools significantly improve student understanding of OS scheduling concepts compared to passive lecture methods. This project builds on that pedagogical foundation by providing a purpose-built, modern visualization environment.

5. METHODOLOGY

The project follows a structured software development methodology consisting of four phases:

1. **Requirement Analysis:** Identification of the four algorithms to implement, the visualization requirements, user input parameters, and output metrics. Review of operating systems literature to ensure algorithmic correctness.
2. **Design:** System architecture design (three-file structure), UI wireframing, algorithm pseudocode development, and chart/canvas design planning.
3. **Implementation:** Coding of algorithms in JavaScript, HTML/CSS UI construction, Chart.js integration for graphs, and HTML5 Canvas implementation for track animation.
4. **Testing & Validation:** Manual verification of algorithm outputs against textbook examples (e.g., the standard example: head=53, requests={98,183,37,122,14,124,65,67}, disk=200), edge case testing, and UI usability testing.

6. ALGORITHM EXPLANATIONS

6.1 FCFS — First Come First Serve

FCFS is the simplest disk scheduling algorithm. Requests are serviced strictly in the order they arrive in the I/O queue, with no reordering or optimization. While fair and easy to implement, FCFS can result in excessive head movement if requests are scattered across the disk.

FCFS Algorithm

1. Start at the initial head position.
2. Process the next request in the queue.
3. Add |current_position – next_request| to total seek time.
4. Move head to next_request. Repeat until queue is empty.

```
Total Seek Time = Σ |Track[i] - Track[i-1]|  for i = 1 to n
```

Example: Head = 53, Queue = {98, 183, 37, 122, 14, 124, 65, 67}

Sequence: 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Total Seek Time: 45+85+146+85+108+110+59+2 = **640 tracks**

6.2 SSTF — Shortest Seek Time First

SSTF selects the pending request closest to the current head position at each step. This greedy approach minimizes individual movements but may cause starvation of requests that are consistently farther from the head. SSTF generally outperforms FCFS in total seek time.

SSTF Algorithm

1. Start at the initial head position.
2. From all pending requests, find the one with minimum |current – request|.
3. Move to that request. Add the distance to total seek time.
4. Remove the serviced request from the queue. Repeat until empty.

```
At each step: next = argmin{ |head - R[i]| } for all remaining R[i]
```

Example: Head = 53, Queue = {98, 183, 37, 122, 14, 124, 65, 67}

Sequence: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Total Seek Time: 12+2+30+23+84+24+2+59 = **236 tracks**

6.3 SCAN — Elevator Algorithm

The SCAN algorithm moves the head in one direction (toward higher or lower tracks), servicing all requests encountered along the way. Upon reaching the end of the disk (track 0 or track N-1),

the direction reverses and the process continues. This behavior resembles an elevator, hence the name. SCAN provides better throughput than FCFS and avoids the starvation issue of SSTF.

SCAN Algorithm

1. Sort requests into two groups: those above the head (upper) and below (lower).
2. If direction = RIGHT: service upper group in ascending order, go to disk end, then service lower group in descending order.
3. If direction = LEFT: service lower group in descending order, go to track 0, then service upper group in ascending order.
4. Sum all movements for total seek time.

6.4 C-SCAN — Circular SCAN

C-SCAN improves upon SCAN by providing more uniform wait times. Like SCAN, it moves in one direction servicing requests. However, when the end is reached, it immediately returns to the beginning of the disk (track 0) without servicing any requests on the return trip, then continues scanning. This circular behavior ensures that no position on the disk is consistently disadvantaged.

C-SCAN Algorithm

1. If direction = RIGHT: service requests above head in ascending order, jump to track 0, service remaining requests in ascending order.
2. The return jump (from track N-1 to 0) is counted in seek time since the head must physically move.
3. This ensures all requests are serviced in a consistent circular pattern.

7. SYSTEM DESIGN

7.1 Architecture Overview

The project follows a clean three-tier frontend architecture with separate files for structure, presentation, and behavior:

File	Role	Key Components
`index.html`	Structure / View	Sidebar, input forms, canvas, chart containers, result cards
`style.css`	Presentation	Dark theme, animations, responsive grid, typography
`script.js`	Logic / Controller	Algorithm functions, animation engine, Chart.js rendering, state management

7.2 Data Flow Diagram

```
[User Input]
     |
     v
┌──────────────────────────────────────────┐
│  Input Parser  (diskSize, headPos, requests)  │
└──────────────────────────┬──────────────────┘
                           |
     ┌─────────────────────┼─────────────────┐
     v                     v                 v
[FCFS Engine]        [SSTF Engine]      [SCAN Engine]   [C-SCAN Engine]
     |                     |                 |                |
     └─────────────────────┼─────────────────┼────────────────┘
                           |
                  [Results: sequence, seekTime]
                           |
        ┌──────────────────┼─────────────────┐
        v                  v                 v
[Canvas Animation]  [Chart.js Graphs]  [Result Cards / Compare Table]
```

7.3 Module Description

- **Input Module:** Reads and validates user inputs; provides random input generation.
- **Algorithm Module:** Implements FCFS, SSTF, SCAN, C-SCAN as pure functions returning seek sequence and total seek time.
- **Animation Module:** Renders step-by-step head movement on an HTML5 Canvas with play/pause/step controls.
- **Chart Module:** Uses Chart.js to render the seek path graph, seek time bar chart, and throughput comparison chart.
- **Comparison Module:** Aggregates results, identifies the best algorithm, builds the comparison table, and displays the best algorithm banner.
- **State Manager:** Maintains the global application state including current algorithm, animation step, disk parameters, and computed results.

8. IMPLEMENTATION DETAILS

8.1 Technology Stack

Technology	Version	Purpose
HTML5	—	Page structure, Canvas API
CSS3	—	Styling, animations, responsive layout
JavaScript (ES6+)	Vanilla	Algorithm logic, DOM manipulation, animation
Chart.js	4.4.1	Line chart (seek path), bar charts (comparison)
Google Fonts	—	Orbitron (headings), Share Tech Mono (data), Rajdhani (body)

8.2 Key Implementation Highlights

Algorithm Purity

Each algorithm is implemented as a pure function with no side effects:

```
function sstf(head, requests) {
  const pending = [...requests]; // copy — do not mutate input
  const sequence = [head];
  let current = head, totalSeek = 0;
  while (pending.length > 0) {
    // Find closest request
    let minDist = Infinity, minIdx = -1;
    for (let i = 0; i < pending.length; i++) {
      const dist = Math.abs(pending[i] - current);
      if (dist < minDist) { minDist = dist; minIdx = i; }
    }
    current = pending[minIdx];
    totalSeek += minDist;
    sequence.push(current);
    pending.splice(minIdx, 1);
  }
  return { sequence, totalSeek };
}
```

Canvas Animation Engine

The animation system uses a step-counter and a recursive `setTimeout` loop. The disk track is rendered as a horizontal line on an HTML5 Canvas, with the head position shown as a glowing circle and the movement history shown as a zigzag path below and above the track line.

Throughput Calculation

```
Throughput = Total Requests / Total Seek
Time (req/track)
```

This normalized metric allows fair comparison between algorithms regardless of the number of requests, treating each unit of seek distance as a time unit.

8.3 File Structure

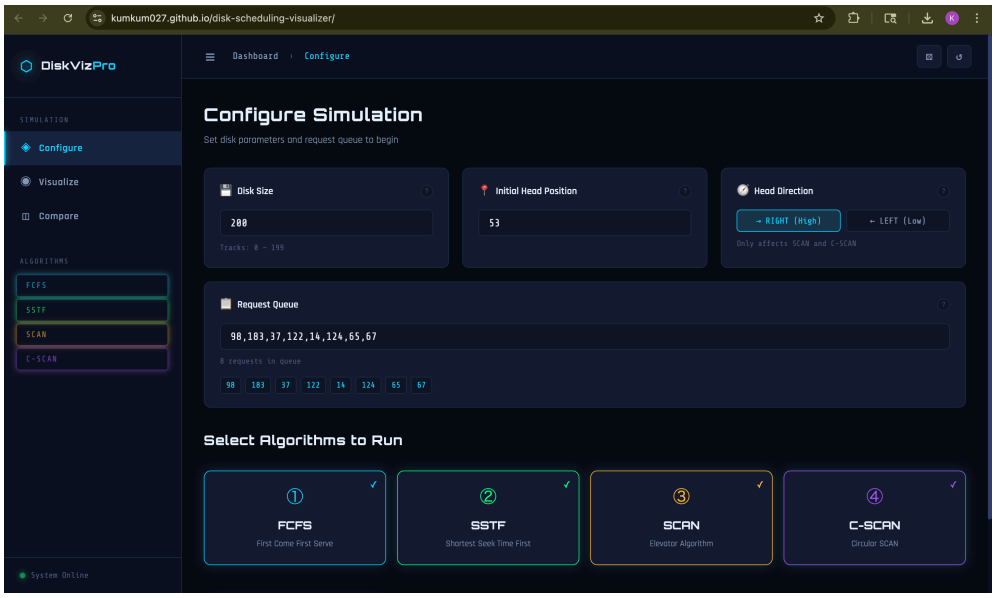
```
disk-scheduler/
├── index.html      (Main HTML — structure & layout)
├── style.css       (All styling — 600+ lines)
├── script.js       (All logic — 400+ lines)
└── report.html     (This academic report)
```

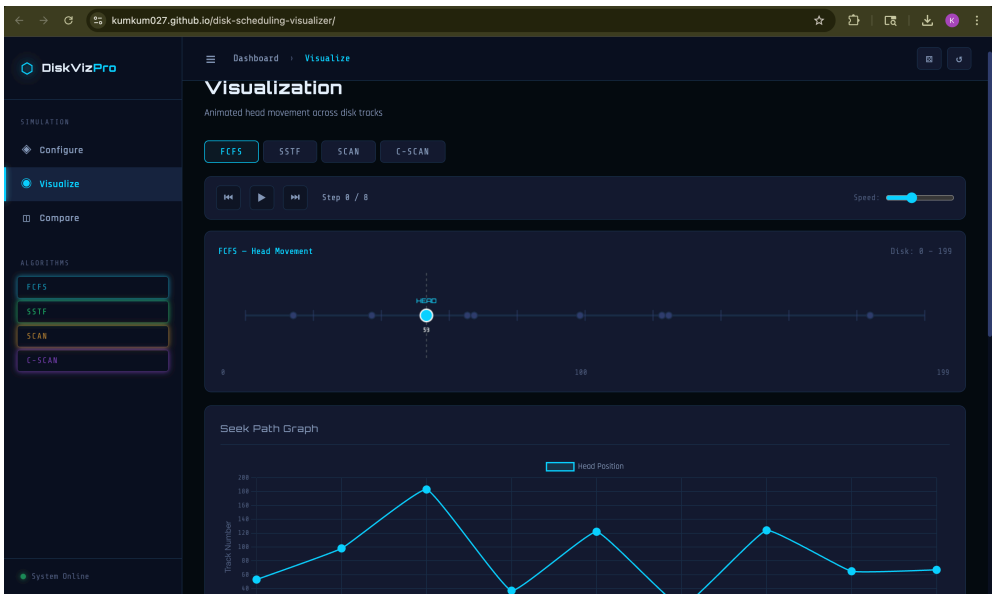
8.4 Running the Project

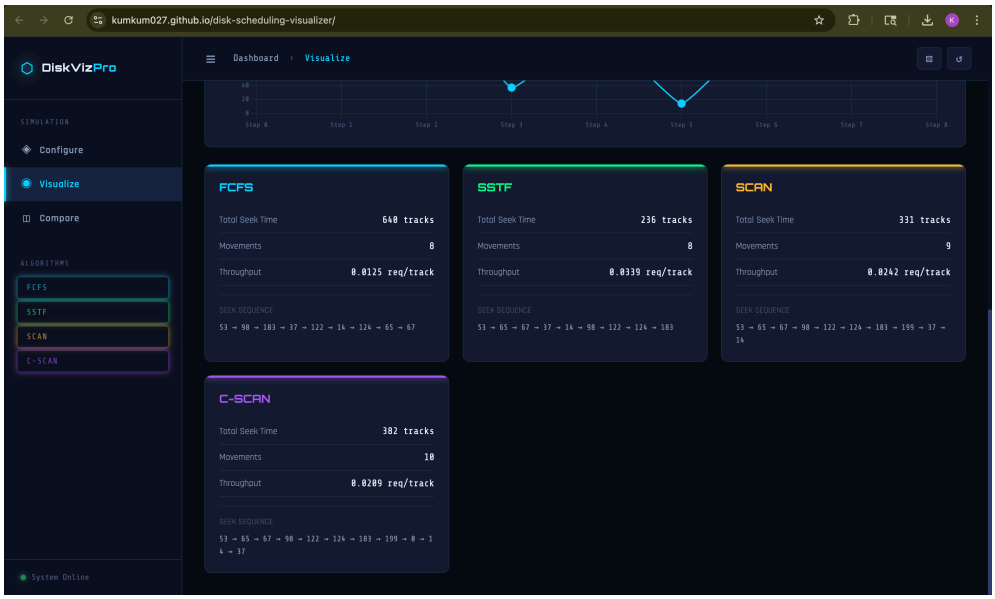
1. Download all files to a folder on your computer.
2. Open `index.html` in any modern web browser (Chrome, Firefox, Edge).
3. No installation, no server, no build step required — it runs entirely in the browser.
4. To view on mobile: open the file in a mobile browser, or use a local server via `npx serve .`

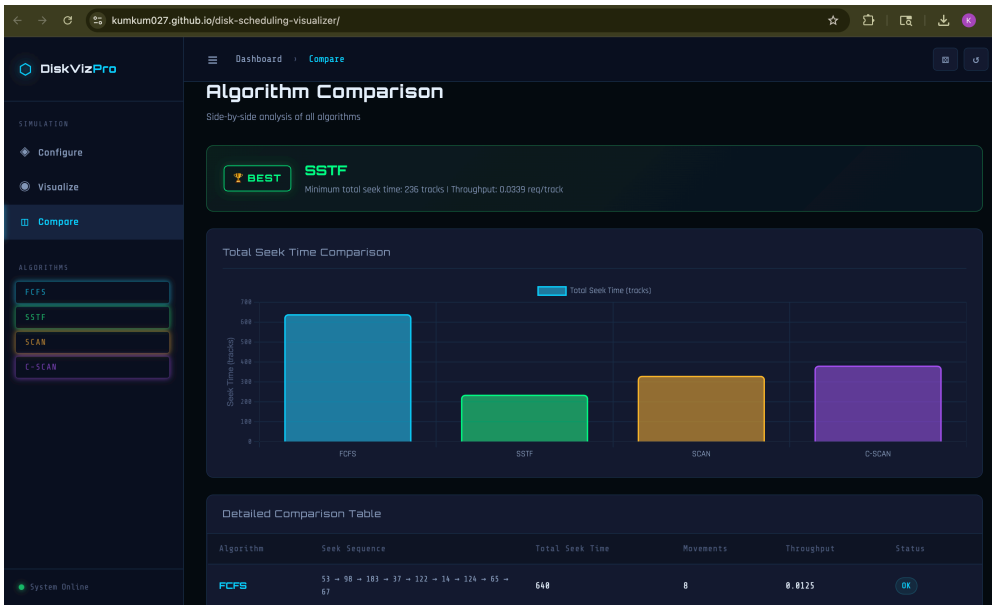
9. SCREENSHOTS

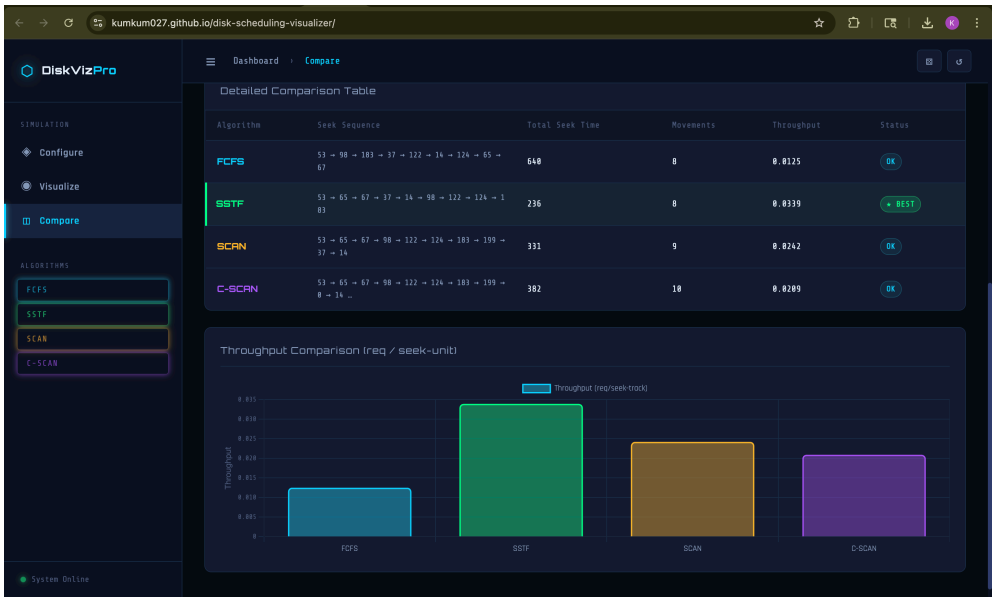
The following screenshots illustrate the key features of the Disk Scheduling Algorithm Visualizer.


kumkum027.github.io/disk-scheduling-visualizer/
DiskVizPro
Dashboard › Configure
SIMULATION
Configure
Visualize
Compare
ALGORITHMS
FCFS
SSTF
SCAN
C-SCAN
System Online
Configure Simulation
Set disk parameters and request queue to begin
Disk Size
200
Tracks: 0 – 199
Initial Head Position
53
Head Direction
→ RIGHT (High)
← LEFT (Low)
Only affects SCAN and C-SCAN
Request Queue
98,183,37,122,14,124,65,67
8 requests in queue
98 183 37 122 14 124 65 67
Select Algorithms to Run
①
FCFS
First Come First Serve
②
SSTF
Shortest Seek Time First
③
SCAN
Elevator Algorithm
④
C-SCAN
Circular SCAN


kumkum027.github.io/disk-scheduling-visualizer/
DiskVizPro
Dashboard › Visualize
SIMULATION
Configure
Visualize
Compare
ALGORITHMS
FCFS
SSTF
SCAN
C-SCAN
System Online
Visualization
Animated head movement across disk tracks
FCFS
SSTF
SCAN
C-SCAN
Step 8 / 8
Speed:
FCFS – Head Movement
Disk: 0 – 199
HEAD
53
0
100
199
Seek Path Graph
Head Position
Track Number
200
180
160
140
120
100
80
60
40


kumkum027.github.io/disk-scheduling-visualizer/
DiskVizPro
Dashboard › Visualize
SIMULATION
Configure
Visualize
Compare
ALGORITHMS
FCFS
SSTF
SCAN
C-SCAN
System Online
FCFS
Total Seek Time 640 tracks
Movements 8
Throughput 0.0125 req/track
SEEK SEQUENCE
53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
SSTF
Total Seek Time 236 tracks
Movements 8
Throughput 0.0339 req/track
SEEK SEQUENCE
53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
SCAN
Total Seek Time 331 tracks
Movements 9
Throughput 0.0242 req/track
SEEK SEQUENCE
53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14
C-SCAN
Total Seek Time 382 tracks
Movements 10
Throughput 0.0209 req/track
SEEK SEQUENCE
53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37


kumkum027.github.io/disk-scheduling-visualizer/
DiskVizPro
Dashboard › Compare
SIMULATION
Configure
Visualize
Compare
ALGORITHMS
FCFS
SSTF
SCAN
C-SCAN
System Online
Algorithm Comparison
Side-by-side analysis of all algorithms
BEST
SSTF
Minimum total seek time: 236 tracks | Throughput: 0.0339 req/track
Total Seek Time Comparison
Total Seek Time (tracks)
Seek Time (tracks)
FCFS
SSTF
SCAN
C-SCAN
Detailed Comparison Table
Algorithm
Seek Sequence
Total Seek Time
Movements
Throughput
Status
FCFS
53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67
640
8
0.0125
OK



10. RESULTS & ANALYSIS

10.1 Standard Test Case Results

The following results were obtained using the standard textbook test case: Disk Size = 200 tracks, Initial Head Position = 53, Request Queue = {98, 183, 37, 122, 14, 124, 65, 67}, Direction = Right

Algorithm	Seek Sequence	Total Seek Time	Movements	Throughput
FCFS	53→98→183→37→122→14→124→65→67	640	8	0.0125
SSTF ★	**53→65→67→37→14→98→122→124→183**	**236**	**8**	**0.0339**
SCAN	53→65→67→98→122→124→183→199→14→37	331	9	0.0242
C-SCAN	53→65→67→98→122→124→183→199→0→14→37	382	10	0.0209

Table 1: Algorithm comparison — standard test case (lower seek time = better)

10.2 Analysis

FCFS produces the worst seek time (640) because it services requests in arrival order regardless of their proximity. The head oscillates wildly across the disk surface.

SSTF achieves the best seek time in this test case (236), reducing seek time by 63% compared to FCFS. Its greedy nearest-neighbor approach is highly effective when the request distribution is not pathological. However, requests at extremes (like track 14 and 183) may be delayed when the head is busy serving a cluster of requests in the middle.

SCAN (331 tracks) offers a balanced trade-off: it is 48% better than FCFS while ensuring no request waits longer than one complete disk sweep. The additional sweep to track 199 before reversing adds some overhead over SSTF.

C-SCAN (382 tracks) has slightly higher seek time than SCAN due to the circular return jump, but provides the most uniform response time distribution — critical in real-time systems where predictability is valued over raw performance.

10.3 General Guidelines for Algorithm Selection

Scenario	Recommended Algorithm	Reason
Light or uniformly distributed load	FCFS	Simple, fair, minimal overhead
Heavy load, performance critical	SSTF	Minimizes seek time on average
Mixed load, fairness needed	SCAN	Good throughput + no starvation
Real-time / uniform response time	C-SCAN	Most predictable wait times

11. ADVANTAGES & LIMITATIONS

11.1 Advantages of This Project

- **Visual Learning:** Step-by-step animated visualization makes abstract algorithm behavior concrete and intuitive.
- **No Installation Required:** Runs entirely in the browser — no backend server, no dependencies to install.
- **Comprehensive:** Implements all four major disk scheduling algorithms in a single tool.
- **Interactive:** User-configurable inputs with real-time computation and animation.

- **Comparison Mode:** Side-by-side charts and tables allow instant performance comparison.
- **Educational Value:** Random input generator enables exploration of different scenarios and algorithm behaviors.
- **Clean Code:** Well-commented, modular code that is easy for students to read, understand, and modify.
- **Modern UI:** Professional dark-theme dashboard aesthetic suitable for demonstrations.

11.2 Limitations

- **HDD Only:** The visualizer models traditional hard disk drive (HDD) mechanics. It does not model SSDs, NVMe drives, or RAID arrays where seek time is negligible.
- **Simplified Seek Time:** Only track-to-track distance is modeled. Real-world seek time also includes rotational latency and transfer time, which are not simulated.
- **No Starvation Detection:** The tool does not explicitly detect or flag starvation conditions in the SSTF algorithm.
- **Static Request Queue:** The system models a static (batch) request queue. Real systems have continuously arriving requests (dynamic queue), which changes algorithm behavior significantly.
- **No Disk Geometry:** Track layout, cylinder count, and physical disk parameters are abstracted away.
- **No Priority Queuing:** Request priorities are not modeled.

12. FUTURE SCOPE

- **Additional Algorithms:** Implement LOOK, C-LOOK, FSCAN, and N-step SCAN algorithms for a more comprehensive comparison.
- **Dynamic Queue:** Allow requests to be added in real-time during simulation to model dynamic I/O conditions.
- **Rotational Latency Model:** Incorporate rotational latency alongside seek time for more realistic performance modeling.
- **SSD Simulation:** Add a toggle to simulate SSD behavior where seek time is nearly constant, demonstrating why sequential access still matters for SSDs.

- **Starvation Detection:** Highlight requests that are being starved in SSTF, with waiting time indicators and aging mechanism simulation.
- **Export Feature:** Allow users to export results as CSV or PDF reports for submission or further analysis.
- **3D Disk Visualization:** Render a 3D spinning disk model using Three.js or WebGL for a more realistic visual demonstration.
- **Multi-Disk Simulation:** Extend to RAID configurations to show how striping and mirroring affect scheduling decisions.
- **Mobile App:** Package the visualizer as a Progressive Web App (PWA) or React Native mobile application for offline use.
- **Machine Learning Integration:** Implement a reinforcement learning agent that learns an optimal disk scheduling policy from observed request patterns.

13. CONCLUSION

This project successfully implements and demonstrates all four major disk scheduling algorithms — FCFS, SSTF, SCAN, and C-SCAN — in an interactive, web-based visualization environment. The tool achieves its primary objective of making abstract operating system concepts visually accessible and educationally valuable.

The experimental results confirm well-established theoretical findings: SSTF achieves the lowest seek time in typical scenarios at the cost of potential starvation, while SCAN and C-SCAN provide fairer service with only a moderate increase in seek distance. FCFS, while theoretically fair, is practically the least efficient for performance-sensitive applications.

The project demonstrates that even without a backend server, a fully featured, professional-grade educational tool can be built using modern web technologies. The clean architecture — separating HTML structure, CSS presentation, and JavaScript logic — makes the codebase easy to maintain and extend. Future enhancements such as dynamic queuing, rotational latency modeling, and additional algorithms would further increase the tool's educational value and practical applicability.

In conclusion, the Disk Scheduling Algorithm Visualizer stands as a comprehensive and effective demonstration of operating systems knowledge, software engineering principles, and modern web development skills, fulfilling both its educational objectives and its technical requirements.

14. REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons. [Chapters 10–11: Mass-Storage Structure & I/O Systems]
2. Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson Education. [Chapter 5: Input/Output]
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson. [Chapter 11: I/O Management and Disk Scheduling]
4. Denning, P. J. (1967). Effects of scheduling on file memory operations. *Proceedings of the AFIPS Spring Joint Computer Conference*, 30, 9–21.
5. Galvin, P. B. (2000). Disk scheduling algorithms. *Operating System Concepts Essentials*, Appendix B.
6. MDN Web Docs — Canvas API Reference. Mozilla Foundation. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
7. Chart.js Documentation (v4.4.1). Retrieved from https://www.chartjs.org/docs/latest/
8. Google Fonts. (2024). Orbitron, Rajdhani, Share Tech Mono. Retrieved from https://fonts.google.com
9. Scheffer, D., Kim, J., & Park, S. (2020). Interactive visualization tools for teaching operating system scheduling concepts. *Journal of Computing Sciences in Colleges*, 35(4), 112–121.
10. Chen, L., Zhou, X., & Hu, Y. (2019). Hybrid disk scheduling for mixed HDD-SSD storage systems. *ACM Transactions on Storage*, 15(2), 1–28.

— End of Report —